

What each script in py5DED does

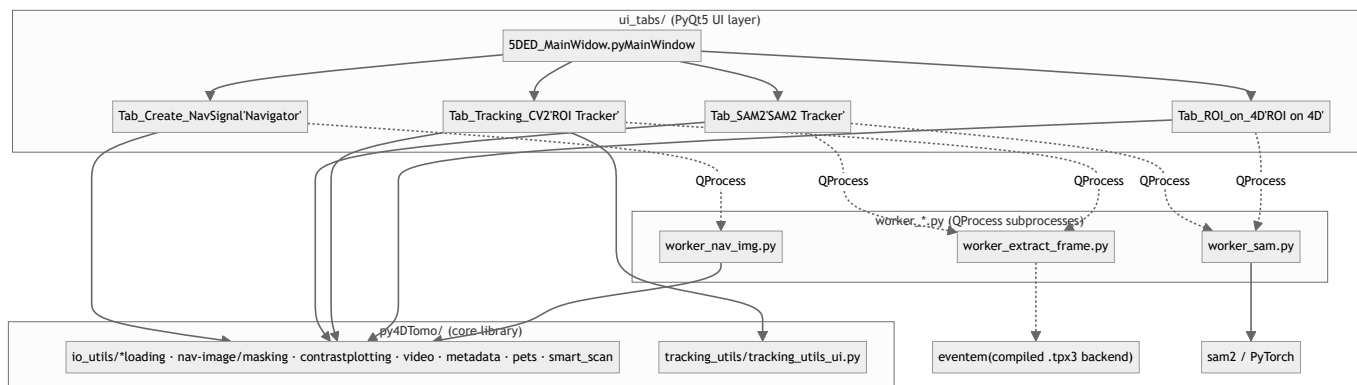
A file-by-file map of the codebase — purpose, functions, and which tab(s) actually call each one — gathered directly against the current working tree so it reflects this session's uncommitted changes, not just the last commit.

Scope — root scripts, py4DTomo/, ui_tabs/

Excluded — other_scripts/ (personal scripts + vendored SAM2 source)

Gathered — 2026-08-07

HOW THE PIECES FIT TOGETHER



ui_tabs/ calls into **py4DTomo** directly for anything fast enough to run in-process (optionally backgrounded on a `QThreadPool`), and dispatches to a **worker_*.py** subprocess via `QProcess` for anything CPU/GPU-heavy or that benefits from real process isolation — batch nav-image calculation, 3DED frame extraction, SAM2 segmentation. Workers launch identically in dev (`python 5DED_MainWidow.py --worker <name> <args>`) and in a frozen build (the exe re-invokes itself with the same flag) — see `worker_launch.py` / `worker_dispatch.py`.

01 Root entry & worker scripts

5DED_MainWidow.py

The app's actual entry point. Defines `MainWindow` (a `QMainWindow` wrapping a `QTabWidget` with the 4 main tabs) and the dark-theme stylesheet. `__main__` sets high-DPI Qt attributes, creates the `QApplication`, installs the crash-log excepthook, builds `MainWindow`, repositions it onto whichever screen the mouse is on (fixes a multi-monitor bug where a fresh window could land off-screen), and starts the event loop. Also the **worker-subprocess dispatch front door**: before any GUI import, it checks for `sys.argv[1] == '--worker'` and hands off to `worker_dispatch.run_worker( ... )` instead of starting the GUI.

CLASSES `MainWindow.init_ui` `MainWindow.closeEvent`

NOTE A 5th tab, `Tab_Converter`, is instantiated on a commented-out line – not currently reachable.

worker_dispatch.py

Receiving side of the `--worker <name> <args>` protocol. Maps a worker name (`nav_img` / `extract_frame` / `sam`) to its script and runs it via `runpy.run_path`, so each worker's own unmodified `if __name__ / sys.argv` parsing works identically whether invoked as a plain script (dev) or via the frozen exe's re-invocation (packaged).

FUNCTIONS `_base_dir` `run_worker`

USED BY `5DED_MainWidow.py's --worker front door`

worker_nav_img.py

Subprocess entry point that computes one navigation image for one 4D-STEM file (via `py4DTomo.io_utils`'s `calculate_nav_img` / `calculate_nav_img_masked`), saves it as a temp `.npy`, and prints the file path to stdout – chosen over base64+pickle to avoid IPC cost with many concurrent workers.

FUNCTIONS `_read_scansize_hdf5` `calculate_nav_img_worker`

USED BY `Navigator`

worker_extract_frame.py

Subprocess entry point that extracts one masked/summed diffraction pattern (a 3DED frame) from a 4D-STEM file, across all 4 supported formats. Result-passing is base64+pickle to stdout, not a file path. Also imported directly as a plain module (`load_dp`) for in-thread, non-subprocess use.

FUNCTIONS

`extract_3ded_mask_single_frame`

`load_dp`

`load_tpx3`

`load_hdf5`

`load_hs`

`load_mib`

`get_scan_size_mib_hdr`

AS SUBPROCESS

SAM2 Tracker

ROI Tracker

LOAD_DP DIRECT

Navigator

SAM2 Tracker

ROI Tracker

ROI on 4D

worker_sam.py

Subprocess entry point that runs SAM2 segmentation – `'image'` (single-frame, point/box-prompted) or `'video'` (cross-frame propagation). Reads input from a pickled `seg_input.pkl`, saves mask(s) as `.npz`, prints a JSON `{'path', 'idx'}` result.

FUNCTIONS

`check_torch_device`

`delete_model`

`__main__` image/video branches

USED BY

SAM2 Tracker (image + video)

ROI on 4D (image only)

tpx3 converter.py

tpx3 converter_multiprocess.py

UNUSED

Standalone, non-GUI scripts that batch-convert raw `.tpx3` event files to dense `.hdf5` via `eventem.FourD8/16/32` – the `_multiprocess` variant parallelizes with `multiprocessing.Pool`. Both use **hardcoded input/output paths** edited directly in the script, not CLI args.

FUNCTIONS

`process_file`

`delete_existing`

USED BY

nobody – not imported anywhere

02 py4DTomo/ — core computational & IO library

py4DTomo/___init___.py is empty – everything lives in the two subpackages below. Both io_utils/___init___.py and tracking_utils/___init___.py use a **wildcard re-export shim**: io_utils/___init___.py does from .io_utils_ui import *, and io_utils_ui.py itself does from py4DTomo.io_utils.
<module> import * for every module below – so import py4DTomo.io_utils as io flattens every public name from every submodule onto io.<name>. This exists because worker_nav_img.py imports io_utils_ui.py as a bare top-level module, which a relative import can't survive.

io_utils/loaders.py

Format-dispatching lazy/eager 4D-STEM loaders for .tpx3 / .hdf5 / .hspy / .zspy / .mib , plus header readers for scan/detector size and smart-scan sparse-frame reconstruction.

FUNCTIONS

get_4d_files load_signal load_tpx3 load_hdf5 load_hs _reconstruct_smart_scan
load_mib get_scan_size get_det_size get_scan_size_mib_hdr

USED BY

All 4 tabs

worker_extract_frame.py

worker_nav_img.py

io_utils/nav_image.py

Navigation-image and diffraction-pattern computation – virtual-detector masks (including multi-detector support), beam-center finding, and format-dispatching nav-image/summed-DP functions (eventem fast path for .tpx3 , dask fallback otherwise).

FUNCTIONS

create_nav_signal_from_haadf calculate_nav_img_tpx3 get_roi get_dp
create_virtual_detector create_virtual_detector_multi find_dp_center
find_dp_center_blurred get_sum_dp calculate_nav_img_masked calculate_nav_img_hdf5
calculate_nav_img

USED BY

Navigator (primary)

worker_nav_img.py

SAM2 / ROI Tracker / ROI-on-4D (find_dp_center_blurred only)

io_utils/contrast.py

8-bit contrast normalization for display/tracking prep, plus binary-mask edge-band erosion and directional single-pixel grow/shrink, used by the mask fine-tune dialog and edge-detection features.

FUNCTIONS

`_contrast_bounds` `convert_to_8bit` `convert_img_to_8bit` `gaussian_blur`
`threshold_ignore_zero` `_directional_kernel` `erode_mask_edge` `shift_mask_edge`

USED BY

`SAM2 Tracker` `ROI Tracker` `Navigator` `MaskEditDialog` `ContrastScalingBox`

io_utils/plotting.py

matplotlib plot-overlay helpers – a readable (background-free, outlined-text) scale bar, and reciprocal-space calibration rings for diffraction-pattern axes.

CLASSES/FNS

`ReadableScaleBar` `add_readable_scalebar` `remove_scalebar` `draw_reciprocal_scale_circles`

USED BY

`All 4 tabs`

io_utils/video.py

Frame/video export – per-frame TIFF stacks and animated clips (DP, tracking-ROI, or SAM2-mask overlays), FFmpeg-piped with a GIF fallback.

FUNCTIONS

`convert_to_rgb` `create_tracking_result` `create_array_from_dissimilar_imgs` `create_frames`
`create_clip_dp` `create_clip_tracking` `create_clip_tracking_with_mask`

USED BY

`All 4 tabs' Save Results`

io_utils/metadata.py

Parses tpx3 acquisition `comment.txt` metadata (scan size, dwell time), and reads/writes the Navigator's saved-analysis sidecar files (`analysis_info.json` , `metadata.json`).

FUNCTIONS

`_resolve_metadata_fn` `get_metadata` `get_metadata_block_count` `default_analysis_save_dir`
`save_analysis_info` `load_analysis_info` `load_analysis_metadata`

USED BY

All 4 tabs (Load Metadata)

SAM2 / ROI Tracker (analysis_info)

io_utils/pets.py

PETS2 (`.pts2`) project-file generation for "Make *.pts2" — electron-wavelength calc, a shared parameter catalogue keeping dialog and generator in sync, cached defaults, and the file writer.

FUNCTIONS

`electron_wavelength_angstrom` `load_pets2_defaults` `save_pets2_defaults` `write_pts2`
`AUTOTASK_OPTIONS` `BASIC_FIELDS` `ADVANCED_FIELDS`

USED BY

Pets2ParamsDialog

SAM2 Tracker

ROI Tracker

io_utils/progress.py

Progress/console plumbing — routes dask's progress bar, and raw OS-fd stdout/stderr from compiled extensions (eventem) that bypass `sys.stdout` , through a Python logger instead of the terminal.

CLASSES/FNS

`format_duration_hms` `_NullWriter` `LoggingProgressBar` `_log_or_print` `_capture_fd`
`redirect_console_to_logger`

USED BY

Internally by `loaders.py` / `nav_image.py`

io_utils/smart_scan.py

Smart-scan (sparse, pattern-file-driven) acquisition support – matches a folder's detection/acquisition/pattern files by tilt angle and flags mismatches for review.

FUNCTIONS

detection_required parse_angle _group_by_angle _nearest_at_angle
_pick_non_acquisition_haadf _dirs_differ match_tilt_files resolve_smart_scan_files

USED BY

SmartScanCheckDialog

All 4 tabs' "Smart Scanned" logic

tracking_utils/tracking_utils_ui.py

MOSTLY UNUSED

OpenCV/cross-correlation ROI tracking, mask-based 3DED extraction, and manual ROI-selection/threshold utilities, built for a CV2-based tracking workflow.

FUNCTIONS

select_roi select_rois select_rois_manual track_roi_cv2 ✓ _xcorr_context_window
_xcorr_bandpass_normalize track_roi_xcorr convert_roi_to_int track_roi
subtract_image_background extract_3ded extract_3ded_mask_single_frame check_threshold
create_masks ✓ cut_imgs_by_roi ✓ translate_roiInRoi ✓ make_tracking_signal
make_tracking_image

USED BY

ROI Tracker only — 4 of 18 fns (✓ above)

SHARED BASE & INFRASTRUCTURE

base_tab.py

`TabBase(qtw.QWidget)` — the shared base every main tab subclasses: logger + optional per-tab `QThreadPool` + the `_cancelling` / `cancel_running_work()` / `cleanup()` convention, centralizing boilerplate that used to be hand-copied 4 times. Plus two module-level helpers: `compute_left_panel_width()` (screen-scaled left-panel width) and `get_existing_directory(parent, caption, start_dir='')` — a folder picker that shows files while browsing, unlike the default `QFileDialog.getExistingDirectory`, so you can confirm a folder's contents before selecting it.

USED BY

All 4 tabs (TabBase)

All 4 tabs + tab_converter.py (get_existing_directory, every folder-browse button)

worker_thread.py

`WorkerThread_General` / `WorkerSignals` — generic `QRunnable` + signals wrapper for `QThreadPool` jobs (`results` / `error` / `finished`, with explicit exception→`error` conversion since Qt otherwise silently swallows exceptions at the `QRunnable` boundary). Plus `ProcessStderrBuffer` — shared `QProcess` stderr handling, since eventem/SAM2 write routine progress there even on success.

USED BY

All 4 tabs

worker_launch.py

`worker_command(worker_name, args)` builds the (`program`, `arguments`) pair for launching a `worker_*.py` script as a real `QProcess`, abstracting the dev-vs-frozen invocation shapes.

USED BY

Navigator

SAM2 Tracker

ROI Tracker

ROI on 4D

logging_utils.py

`QtLogHandler` / `get_qt_log_handler` / `get_tab_logger` route Python logging into the Qt log console via a thread-safe signal; `LogConsole` is the embedded widget each tab has its own copy of; `install_excepthook` routes uncaught exceptions to the log instead of vanishing silently.

USED BY

All 4 tabs

5DED_MainWidow.py (install_excepthook)

THE 4 MAIN TABS

`tab_create_navSignal.py` → `Tab_Create_NavSignal` "Navigator" · ~45 methods

Builds navigation-image signals from a folder of raw 4D-STEM files. No persistent DataFrame – state is a file list plus `self._analysis_metadata`.

KEY METHODS

`calculate_button` `create_navigation_signal` `launch_next_nav_task` `test_selected_file`
`compute_sum_dp*` `add/remove_extra_detector` `get_active_detectors`
`on_press/motion/release_mask` `open_smart_scan_check_dialog` `save_results`

`tab_sam2.py` → `Tab_SAM2` "SAM2 Tracker" · ~68 methods · largest data model

SAM2 point/box-prompted tracking.

`self.df_obj`: use, idx, frame_idx, points, labels, end, single_mask, mask, mask_default, rois, dp – one row per tracked object.

KEY METHODS

`load_navSignal` / `load_saved_analysis` `on_click` `initiate_video_segmentation`
`run_video_segmentation` `launch_next_video_seg` `initiate_image_segmentation` `extract_3ded`
`extract_dp_current_frame` `duplicate_object` `open_fine_tune_mask_dialog` `save_results`

`tab_tracking_cv2.py` → `Tab_Tracking_CV2` "ROI Tracker" · ~64 methods · largest file in the repo

OpenCV/cross-correlation ROI tracking, with ROI-in-ROI relationships.

`self.df_rois`: use, init, in_rois, end, ref, out_rois, mask, dp – `ref` supports one ROI tracking relative to another.

KEY METHODS

`launch_auto_detector` / `receive_objects` `track_rois` `extract_3ded`
`extract_dp_current_frame` `open_fine_tune_mask_dialog` `save_results`

`tab_roi_4d.py` → `Tab_ROI_on_4D` "ROI on 4D" · ~38 methods · structural outlier

Single-image/single-ROI SAM2 segmentation directly on one 4D-STEM file — no multi-object tracking, no persistent DataFrame. Defines its own local `QRunnable`s` ( `Worker_NavImg` , `Worker_CalculateDP` , `Worker_CalculateDP_Mask` ) rather than reusing `WorkerThread_General` .

KEY METHODS

`get_nav_image` `segment_image` `compute_seg_dp` `get_dp_from_mask` `open_threshold_dialog`
`compute_sum_dp_from_threshold`

`tab_converter.py` → `Tab_Converter` "tpx3 Converter" **DISABLED**

GUI-driven `.tpx3` → `.hdf5` batch conversion — functionally the same `eventem.FourD*` calls as the standalone root scripts, reimplemented as a proper tab. Doesn't subclass `TabBase` and defines its own `WorkerThread_processing` / `Communicator` rather than reusing `worker_thread.py` — reads as legacy/parked code from before the shared-infrastructure refactor. **Currently commented out of MainWindow's tab bar — not reachable in the running app.**

DIALOGS & SHARED WIDGETS

CLASS	FILE	PURPOSE	USED BY
MaskEditDialog	mask_edit_dialog.py	Frame-by-frame mask fine-tuning on a tracked object/ROI's mask stack: Ctrl+Scroll zoom, Ctrl+Click(+drag) pixel paint, Shift+drag region paint, a directional D-pad for 1px grow/shrink, a live non-destructive Edge Detection preview, and "Reset to Tracking".	SAM2 Tracker, ROI Tracker
ThresholdDialog	threshold_dialog.py	"Summed DP from Threshold" popup – check/adjust a real-space threshold mask before committing to the actual summed-DP computation.	Navigator, ROI on 4D
SmartScanCheckDialog	smart_scan_dialog.py	Editable per-tilt-angle table matching detection/acquisition/pattern files for a smart-scanned dataset, with drag-and-drop file-cell swapping.	Navigator, SAM2 Tracker, ROI Tracker
Pets2ParamsDialog / Pets2AutotaskDialog	pets2_dialog.py	Collects PETS2 .pts2 export parameters and the autorun task-list editor; caches last-used values.	SAM2 Tracker, ROI Tracker
ContrastScalingBox	contrast_scaling.py	"Display Contrast" group box (Percentile / Min-Max / Std. Dev.), synchronous single-frame preview + backgrounded full-stack rescale.	SAM2 Tracker, ROI Tracker
Object_Detector_Widget	object_detection_widget.py	Standalone auto-object-detection popup (threshold/morphology-based, debounced live preview).	ROI Tracker only

CLASS	FILE	PURPOSE	USED BY
LoadingSpinner	loading_label.py	Translucent "LOADING..." overlay spinner for long extract/track operations.	SAM2 Tracker, ROI Tracker

Dead / dormant code worth knowing about

- `tab_converter.py` (`Tab_Converter`) – not wired into MainWindow's tab bar; not reachable in the running app.
- `tracking_utils_ui.py` – 14 of its 18 functions (including `extract_3ded` / `extract_3ded_mask_single_frame`) have no live callers anywhere.
- Root `tpx3_converter.py` / `tpx3_converter_multiprocess.py` – hardcoded-path standalone scripts, not imported by anything; superseded in intent (if not reachability) by the disabled `tab_converter.py` .
- `object_detection_widget.py` – contains a sizeable dead/commented-out UI block (`self.group_input` section), suggesting a paused sub-feature.

Gathered by reading the working tree directly (no worktree isolation) so it reflects this session's uncommitted edits. File and function names are exact; "used by" reflects live import/call sites checked via grep, not inferred from naming.